

Labo 03 — Réponses commentées

Question 1 — Trois comportements, une règle

Réponse. La règle : **un conteneur vit exactement le temps de son processus principal.**

- `docker run alpine` : la commande par défaut de l'image est `/bin/sh`. Sans `stdin` attaché, `sh` lit une entrée vide, atteint la fin de fichier et se termine. Le conteneur meurt donc en quelques millisecondes.
- `docker run nginx` : `nginx` est lancé avec `daemon off;`, il tourne au premier plan et ne rend jamais la main. Le conteneur vit tant que `nginx` vit.
- `docker run -it alpine sh` : `-i` maintient `stdin` ouvert et `-t` fournit un terminal. `sh` attend vos commandes, donc ne se termine pas.

Pourquoi. Docker ne surveille rien d'autre que ce processus. Il n'a aucune notion de « service prêt » ou de « conteneur en bonne santé » — cela viendra avec `HEALTHCHECK` (labo 10).

Nuance. L'erreur classique en découle : `docker run -d alpine` puis étonnement de le voir en `Exited (0)`. Rien n'a échoué ; il n'y avait simplement rien à faire.

Exemple.

```
docker run alpine ; docker ps -a --latest --format '{{.Status}}'    # Exited (0)
docker run -d nginx:alpine ; docker ps --format '{{.Status}}'    # Up 2 seconds
```

Question 2 — Le script qui met Java en arrière-plan

Réponse. Le `&` détache `java` du script. Le script exécute alors `echo`, arrive à sa dernière ligne et se termine avec le code `0`. Or ce script **est** le PID 1 : sa fin entraîne la destruction du conteneur, et Java est tué avec lui. Correction : supprimer le `&` et remplacer la dernière ligne par un `exec`.

```
#!/bin/sh
echo "API démarrée"
exec java -jar /app/api.jar
```

Pourquoi. `exec` **remplace** le processus du shell par celui de Java, qui hérite du PID 1. Sans `exec`, le shell resterait PID 1 et intercepterait les signaux sans les transmettre : `docker stop` deviendrait un `SIGKILL` déguisé au bout de 10 secondes.

Nuance. Le réflexe « je vais mettre un `sleep infinity` ou un `tail -f /dev/null` à la fin du script pour qu'il ne se termine pas » est un contresens : le conteneur reste vivant même quand Java a planté, plus aucun redémarrage automatique n'est déclenché, et `docker logs` ne montre plus rien. On fabrique un service mort qui a l'air vivant.

Exemple.

```
docker run --rm alpine sh -c 'sleep 300 & echo lance'    # revient tout de suite
docker run -d --name ok alpine sh -c 'exec sleep 300'    # reste Up
docker rm -f ok
```

Question 3 — `sleep` met 10 secondes et sort en 137

Réponse. Parce que le noyau Linux traite le **PID 1 de façon spéciale** : il n'applique pas l'action par défaut d'un signal si le processus n'a pas installé de gestionnaire pour ce signal. `sleep` ne gère pas `SIGTERM` ; en PID 1, le signal est donc simplement **ignoré**. Docker attend son délai de grâce de 10 secondes, puis envoie `SIGKILL`, qui, lui, ne peut pas être ignoré : d'où le code `137` ($128 + 9$) au lieu de `143` ($128 + 15$).

Pourquoi. Cette protection du noyau existe pour éviter qu'un signal mal placé ne tue l'`init` d'un système et ne provoque un *kernel panic*. Dans un conteneur, elle produit cet effet de bord déroutant.

Nuance. Le même `sleep 300` lancé comme processus **enfant** s'arrête instantanément : ce n'est pas une propriété de `sleep`, c'est une propriété du PID 1. Et le symptôme est pernicieux en production : chaque arrêt coûte 10 secondes, ce qui, multiplié par vingt conteneurs lors d'un redéploiement, fait plusieurs minutes de coupure inexplicquée.

Exemple.

```
docker run -d --name veille alpine sleep 300
time docker stop veille          # real 0m10.1s
docker inspect --format '{{.State.ExitCode}}' veille  # 137
docker rm veille
```

Question 4 — Ce que `--init` change

Réponse. `--init` insère un mini-processus d'`init` (`tini`) comme PID 1. Votre commande devient son **enfant**. N'étant plus PID 1, elle perd la protection spéciale du noyau : `SIGTERM` lui applique donc son action par défaut, la terminaison immédiate. D'où l'arrêt instantané et le code `143`.

Pourquoi. `tini` fait deux choses : il **relaie** les signaux qu'il reçoit à son enfant, et il **récupère** les processus orphelins (`wait()`), empêchant l'accumulation de zombies. C'est le rôle d'un `init`, que votre application n'est pas censée assurer.

Nuance. `--init` n'est pas la solution universelle. Si votre application gère correctement `SIGTERM` — c'est le cas de nginx, de PostgreSQL et de la JVM avec un *shutdown hook* — elle n'en a pas besoin. `--init` est utile pour les processus qui ignorent les signaux, et pour ceux qui engendrent des enfants sans les attendre (scripts, outils Node avec sous-processus). Le vrai correctif de fond reste la forme `exec` dans le Dockerfile.

Exemple.

```
docker run -d --init --name veille alpine sleep 300
time docker stop veille          # real 0m0.1s
docker inspect --format '{{.State.ExitCode}}' veille  # 143
docker rm veille
```

Question 5 — `-i` et `-t` séparément

Réponse. `-i` garde l'entrée standard connectée ; `-t` alloue un pseudo-terminal.

- `docker run -t alpine sh` : vous obtenez un prompt d'apparence normale, mais votre clavier n'est **pas** relié au conteneur. Taper `ls` n'a aucun effet ; le shell attend une entrée qui n'arrivera jamais. Il faut sortir avec `Ctrl+C` depuis un autre terminal ou tuer le conteneur.

- `docker run -i alpine sh` : ça **fonctionne**, mais sans confort : pas de prompt, pas d'écho des caractères tapés, pas de couleurs, pas d'historique. Vous tapez à l'aveugle, et la sortie s'affiche.

Pourquoi. Le prompt, l'écho et l'édition de ligne sont des services rendus par le terminal, pas par le shell. `-t` fournit ce terminal, `-i` fournit le canal.

Nuance. En CI et dans les scripts, `-t` provoque une erreur (`the input device is not a TTY`) car aucun terminal n'existe. La bonne habitude est donc : `-it` pour un humain, `-i` seul dans un script, aucun des deux pour une commande qui ne lit rien.

Exemple.

```
echo "echo bonjour" | docker run -i --rm alpine sh      # affiche : bonjour
docker run -it --rm alpine sh -c 'tty'                 # /dev/pts/0
docker run -i --rm alpine sh -c 'tty'                 # not a tty
```

Question 6 — `Ctrl+C` après un `attach`

Réponse. `docker attach` rebranche votre terminal sur l'entrée/sortie du **PID 1 existant**. Le `Ctrl+C` a donc envoyé `SIGINT` directement au processus principal, qui s'est arrêté — et le conteneur avec lui. Les deux bonnes façons de consulter les logs : `docker logs -f mon-api`, ou, si l'on tient à `attach`, se détacher avec la séquence `Ctrl+P` `Ctrl+Q`.

Pourquoi. `attach` n'est pas un « visualiseur » : c'est un branchement réel sur les flux du processus. Tout ce que vous tapez lui est transmis.

Nuance. `docker logs -f` est non seulement plus sûr, mais plus complet : il rejoue **l'historique depuis le début** (`--tail 100` pour se limiter), alors qu' `attach` ne montre que ce qui arrive à partir de maintenant. On peut aussi neutraliser le risque avec `docker attach --sig-proxy=false`, qui cesse de transmettre les signaux — mais autant utiliser `logs`.

Exemple.

```
docker run -d --name web nginx:alpine
docker logs -f --tail 20 web      # Ctrl+C ici n'arrête QUE l'affichage
docker ps --filter name=web      # toujours Up
docker rm -f web
```

Question 7 — Trois statuts, trois diagnostics

Réponse.

Statut	Hypothèse	Commande de confirmation
Exited (137)	Tué par <code>SIGKILL</code> : dépassement de la limite mémoire (OOM), ou <code>stop</code> ayant dépassé le délai de grâce	<code>docker inspect --format '{{.State.OOMKilled}}' api</code>
Exited (143)	Arrêt propre sur <code>SIGTERM</code> — probablement un <code>docker stop</code> volontaire ou un redéploiement	<code>docker events --since 10m --filter container=worker</code>
Exited (127)	Commande introuvable dans l'image : mauvais <code>ENTRYPOINT</code> , binaire absent, ou chemin erroné	<code>docker logs batch</code> puis <code>docker inspect --format '{{json .Config}}' batch</code>

Pourquoi. Les codes ≥ 128 signalent une mort par signal : le numéro du signal est le code moins 128 (9 = KILL, 15 = TERM, 2 = INT). Les codes 125-127 sont réservés par Docker aux erreurs de lancement, avant même que votre application ne démarre.

Nuance. 137 est ambigu et c'est ce qui le rend piégeux : « tué » ne dit pas « par qui ». `.State.00MKilled` est le seul discriminant fiable. Attention aussi au faux ami : une application qui **choisit** de sortir avec le code 137 produirait le même statut ; c'est rare mais cela existe (scripts qui propagent le code de leur enfant).

Exemple.

```
docker ps -a --format 'table {{.Names}}\t{{.Status}}\t{{.Image}}'
docker inspect --format '{{.State.ExitCode}} OOM={{.State.00MKilled}} err={{.State.Error}}' api
```

Question 8 — Les logs dans un fichier

Réponse. `docker logs` ne lit **que** `stdout` et `stderr` du PID 1. Une application qui écrit dans un fichier n'y apparaît pas : Docker n'a aucune visibilité sur le contenu du système de fichiers du conteneur.

Pourquoi. Le daemon branche des tubes sur les deux sorties standard du processus principal et enregistre ce qui y transite via un *logging driver* (par défaut `json-file`). Tout le reste lui est invisible.

Nuance. Monter `/var/log/api` sur l'hôte « marche », mais reconstruit précisément ce qu'on essayait de fuir : il faut gérer la rotation soi-même, les permissions du volume, la collecte fichier par fichier, et l'on ne peut plus consulter les logs avec les outils de la plateforme. Surtout, cela casse le modèle du conteneur jetable : les logs d'un conteneur supprimé sont perdus ou orphelins. La bonne réponse est de **retirer** `logging.file.name` et de laisser Spring Boot écrire sur la console ; le *logging driver* de Docker se charge du reste et un collecteur (Loki, Fluent Bit, ELK) les centralise. C'est le principe des logs comme flux d'événements, du *12-factor app*.

Exemple.

```
docker run --rm alpine sh -c 'echo "vers stdout"; echo "vers fichier" > /tmp/f.log'
# seul "vers stdout" apparaît dans docker logs
docker inspect --format '{{.HostConfig.LogConfig.Type}}' <conteneur> # json-file
```

Question 9 — `stop` / `start` contre `rm` / `run`

Réponse. `stop` puis `start` : les données sont **conservées**. `rm` puis `run` : elles sont **perdues**.

Pourquoi. La couche d'écriture appartient au conteneur, pas au processus. L'arrêter ne détruit que le processus ; la couche reste sur le disque, avec sa configuration et ses logs. `docker rm` détruit le conteneur **et** sa couche. Le nouveau `docker run` fabrique un conteneur neuf, reparti de l'état exact de l'image.

Nuance. Cette persistance est réelle mais **trompeuse** : elle vous attache à une machine et à un conteneur précis. Un conteneur ainsi « précieux » ne peut plus être redéployé, ni déplacé, ni reconstruit — l'inverse de ce qu'on cherche. Compter dessus est une erreur de conception : ce qui doit survivre doit être dans un **volume** (labo 06).

Exemple.

```
docker run -d --name pg -e POSTGRES_PASSWORD=x postgres:16-alpine
docker exec pg psql -U postgres -c 'CREATE TABLE t(id int);'
docker stop pg && docker start pg && sleep 3
docker exec pg psql -U postgres -c '\dt'      # la table est là
docker rm -f pg                               # ... et maintenant elle ne l'est plus
```

Question 10 — `always` contre `unless-stopped`

Réponse. La différence n'apparaît qu'après un arrêt **manuel** suivi d'un redémarrage du daemon (ou de la machine).

Scénario : vendredi soir, vous faites `docker stop api` pour une maintenance. Le samedi, le serveur redémarre (mise à jour du noyau). - Avec `--restart=always` : le daemon relance `api` au démarrage. Votre maintenance est annulée sans que personne ne l'ait décidé. - Avec `--restart=unless-stopped` : Docker a mémorisé que **vous** l'aviez arrêté ; il le laisse arrêté.

Pourquoi. Docker enregistre l'intention de l'opérateur dans l'état du conteneur (`.State.Restarting`, et le champ interne qui note l'arrêt manuel). `always` l'ignore délibérément, `unless-stopped` la respecte.

Nuance. Pour un service de production sur machine unique, `unless-stopped` est le bon choix. Mais attention : sous un orchestrateur (Kubernetes, Swarm, ou même Compose avec des politiques de déploiement), **on ne met pas de restart policy Docker** — c'est l'orchestrateur qui gère les redémarrages, et les deux mécanismes se marchent dessus. Enfin, aucune politique ne remplace un `HEALTHCHECK` : un conteneur figé mais vivant ne sera jamais redémarré, puisqu'il n'est pas sorti.

Exemple.

```
docker run -d --restart=unless-stopped --name api nginx:alpine
docker inspect --format '{{json .HostConfig.RestartPolicy}}' api
# {"Name":"unless-stopped","MaximumRetryCount":0}
docker rm -f api
```

Question 11 — Retrouver la première tentative

Réponse. `docker logs` conserve la sortie **de toutes les exécutions successives du même conteneur**, dans l'ordre : les logs de la première tentative sont en tête. Utilisez `docker logs mon-conteneur | head -n 50`, ou `docker logs --since / --until` pour cibler. Un `docker restart` n'efface rien, mais il **ajoute** du bruit et, surtout, écrase l'état du diagnostic : `.State.ExitCode`, `.State.FinishedAt` et `.State.OOMKilled` prennent les valeurs de la nouvelle exécution.

Pourquoi. Le fichier de logs (`json-file`) est attaché au **conteneur**, pas à l'exécution : chaque redémarrage y ajoute. En revanche `.State` ne décrit que la dernière exécution en date.

Nuance. Deux limites à connaître. D'abord `docker rm` détruit ces logs définitivement : ne supprimez jamais un conteneur en panne avant d'avoir extrait ce dont vous avez besoin. Ensuite, un conteneur qui boucle peut produire des logs volumineux qui font tourner le fichier (`max-size`) et effacent justement le début — la cause initiale. D'où la centralisation des logs en entreprise.

Exemple.

```
docker logs mon-conteneur | head -n 50 # la cause initiale
docker logs --since 2026-08-17T09:00:00 mon-conteneur
docker inspect --format '{{.RestartCount}} redemarrages' mon-conteneur
```

Question 12 — Du moins au plus intrusif

Réponse.

Rang	Commande	Ce qu'elle apprend	Intrusion
1	<code>docker inspect</code>	Configuration, limites, état, code de sortie précédent	Nulle : lit le daemon
2	<code>docker logs</code>	Ce que l'application a dit, et où elle s'est arrêtée de parler	Nulle : lit un fichier
3	<code>docker stats</code>	CPU, mémoire, E/S réseau et disque en temps réel	Quasi nulle : lit les cgroups
4	<code>docker top</code>	La liste des processus du conteneur, vue de l'hôte	Quasi nulle : lit <code>/proc</code>
5	<code>docker exec</code>	Tout le reste — mais crée un processus dans le conteneur	Réelle

Pourquoi. Les quatre premières interrogent l'hôte et le daemon *au sujet* du conteneur. `exec` entre dedans : il consomme de la mémoire et du CPU dans un conteneur déjà en difficulté, et peut le faire dépasser sa limite mémoire — donc déclencher l'OOM que vous essayiez de diagnostiquer.

Nuance. `docker top` est sous-estimé : sur une API Java qui sature un cœur, il donne immédiatement le PID hôte du thread fautif, exploitable avec les outils de l'hôte (`top -H -p` , `jstack` via `docker exec` seulement en dernier recours). Notez aussi que `exec` échoue sur un conteneur dont le PID 1 est bloqué mais qui n'a pas de shell (image *distroless*) : encore une raison de ne pas en dépendre.

Exemple.

```
docker stats --no-stream mon-api
docker top mon-api
docker inspect --format '{{.State.Status}} {{.HostConfig.Memory}}' mon-api
```

Question 13 — API et base dans le même conteneur

Réponse. Trois conséquences :

1. **Un seul PID 1, donc un seul cycle de vie.** Si PostgreSQL plante, le conteneur ne s'arrête pas (ce n'est pas le processus principal) : vous avez une API vivante branchée sur une base morte, que rien ne redémarre. Inversement, redémarrer l'API impose de couper la base.
2. **Impossible de faire évoluer les deux séparément.** Un correctif sur l'API force la reconstruction et le redéploiement d'une image contenant la base ; on ne peut pas non plus lancer trois instances de l'API — cela ferait trois bases distinctes.
3. **Les données sont piégées.** Le conteneur devient précieux : on ne peut plus le détruire ni le recréer sans perdre la base, ce qui annule le principal bénéfice du modèle.

Pourquoi. La règle « un processus principal par conteneur » n'est pas dogmatique : elle découle mécaniquement du fait que Docker ne surveille que le PID 1.

Nuance. « Un processus » ne veut pas dire « un seul processus système » : nginx a des *workers*, PostgreSQL a des processus enfants, la JVM a des threads. La règle porte sur **un service, une responsabilité, un cycle de vie**. Il existe de rares exceptions légitimes (un side-car de collecte de logs, un `supervisord` dans un contexte hérité), mais elles se paient toujours en complexité d'exploitation.

Exemple.

```
# Le bon modèle : deux conteneurs, deux cycles de vie, un réseau commun (labo 07).
docker network create appnet
docker run -d --name db --network appnet -e POSTGRES_PASSWORD=x postgres:16-alpine
docker run -d --name api --network appnet mon-api:1.0
docker restart api           # la base n'est pas touchée
```

Question 14 — `--rm` en ponctuel, pas en production

Réponse. `--rm` supprime le conteneur dès qu'il sort. Pour une commande ponctuelle (`docker run --rm alpine date`), c'est idéal : rien ne s'accumule. Pour un service, c'est dangereux : au moindre arrêt, **tout le contexte du crash disparaît**.

Pourquoi. La suppression emporte la couche d'écriture, les logs (`docker logs` n'existe plus), et l'état inspectable — code de sortie, `OOMKilled`, `FinishedAt`, `RestartCount`. Vous apprenez qu'un service est tombé, sans aucun moyen de savoir pourquoi.

Nuance. `--rm` est de plus incompatible avec les *restart policies* : Docker refuse `--rm` avec `--restart` (`Conflicting options: --restart and --rm`), ce qui est cohérent — on ne peut pas redémarrer ce qu'on a supprimé. La bonne pratique de production est l'inverse : garder le conteneur arrêté, l'inspecter, puis le remplacer explicitement lors du déploiement suivant. C'est ce que font les orchestrateurs, qui conservent les objets terminés un certain temps précisément pour permettre l'autopsie.

Exemple.

```
docker run --rm -d --restart=always nginx:alpine
# docker: conflicting options: cannot specify both --restart and --rm
```